



Fw: JCA-B1 settlements API and ApplePay

From Sebastiano Serri <sebastiano.serri@starkbuild.co.uk>

Date Sat 19/09/2026 19:42

To Puneet Kumar <ext.puneet.kumar@starkbuild.co.uk>

 1 attachment (44 KB)

SettlementsAPI.yml;

FYI

Sebastiano Serri
Integration Architect

EMAIL sebastiano.serri@starkbuild.co.uk
STARK Building Materials UK Limited



From: Sebastiano Serri <sebastiano.serri@starkbuild.co.uk>

Sent: Saturday, September 19, 2026 19:42

To: Ganesh Waghmare <ext.ganesh.waghmare@starkbuild.co.uk>; Mukesh Kumar Yadav <ext.mukeshkumar.yadav@starkbuild.co.uk>; Sumit Ghosh <ext.sumit.ghosh@starkbuild.co.uk>

Cc: Shakeel Ahmed <shakeel.ahmed@starkbuild.co.uk>; Sanjay Kumar Dalmia <ext.sanjay.kumardalmia@starkbuild.co.uk>

Subject: JCA-B1 settlements API and ApplePay

Hi all,

I've done a bit of due diligence about the ApplePay integration. You can find an updated version of the OpenAPI specifications for the settlements API. I specifically modified the **`/v1/accounts/{accountId}/settlements/{settlementId}/submit`** endpoint.

This was because I did not include any payload that carried the selected payment method. As you can see from the examples, this endpoint now covers payments with card and ApplePay (and in future GooglePay). It also caters for the use of a new card, with the option to save it.

Use with a stored card:

```
{
  "paymentMethod": "CARD",
  "card": {
    "type": "STORED",
    "cardId": "00000000-0000-0000-0000-000000000001"
  }
}
```

Use with a new card, saved:

```
{
  "paymentMethod": "CARD",
  "card": {
    "type": "NEW",
    "saveForFutureUse": true
  }
}
```

Use with a new card, not saved:

```
{
  "paymentMethod": "CARD",
  "card": {
    "type": "NEW",
    "saveForFutureUse": false
  }
}
```

Use with ApplePay:

```
{
  "paymentMethod": "APPLEPAY",
  "token": {
    "paymentData": {
      "version": "EC_v1",
      "data": "Qm9ndXNFbmNyeXB0ZWRCeXRlc0hlcmU9PQ==",
      "signature": "MIAGCSqGSIb3DQEHAqCAMIACAQExDzANBgIghkgBZQMEAgEFAD...",
      "header": {
        "ephemeralPublicKey": "MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE...",
        "publicKeyHash": "xYzAbCdEfGhIjKlMnOpQrStUvWxYz1234567890=",
        "transactionId": "7f3a4e0912ab34cd56ef789012345678"
      }
    }
  },
  "paymentMethod": {
    "displayName": "Visa 1234",
    "network": "Visa",
    "type": "credit"
  }
},
```

```
"transactionIdentifier": "ABCDEF1234567890"
}
```

Processing a card payment remains asynchronous as we discussed, and the response will be:

```
{
  "paymentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "REQUIRES_ACTION",
  "nextAction": {
    "type": "REDIRECT",
    "url": "https://smartpay/..."
  }
}
```

As you can see the API returns a property called `nextAction` to inform the client that it needs to redirect to the SmartPay Hosted Pages.

So, everything remains the same.

With ApplePay, the App will initiate the payment by showing the ApplePay sheet where the user can select the card to use from the wallet and then be authenticated.

Once the payment has been authorised by the user, the app receives an ApplePay Payment Token and passes the complete token in the request payload when calling the **`/v1/accounts/{accountId}/settlements/{settlementId}/submit`** endpoint.

Processing an Apple Pay payment is synchronous: the submit endpoint calls SmartPay directly and waits for the payment to be processed. We should use this approach because, otherwise, we would need to store the Apple Pay token in the database, as it is required for both the `beginWebPayment` and `updateWebPayment` calls. From a security perspective, we should avoid persisting the token. This reduces the risk of the token being exposed in the event of unauthorized access to the database, even though the token expires 48 hours after its creation. For Apple Pay payments, the `authType` should therefore be set to **`AuthAndSettle`**, as authorization and settlement are performed in a single stage. For card payments, remember that we use **`authType = No`**, since the payment is processed asynchronously in two stages.

With ApplePay, the endpoint returns the following response:

```
{
  "paymentId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "PROCESSING",
  "nextAction": null
}
```

And the app still needs to poll to determine the status of the payment.

As you can see, the returned status can be `PROCESSING`. This caters for scenarios where the request is successfully sent to SmartPay but we do not receive the response, for example because of a network or timeout issue. We therefore need to introduce an additional payment state representing a transaction that has been submitted to SmartPay but whose final outcome is not yet known. Under normal circumstances, this state is transitioned to the appropriate final state as soon as we receive the response from SmartPay. If the response is not received, the background process must determine the outcome of the transaction by calling `getWebPayment` and update the payment state accordingly.

Please consider what I just described as a trace to follow/be inspired when working on the two payment stages + ApplePay and note that this architecture is also compatible with GooglePay.

On a different subject, this is more of an architectural and security consideration. I want to make sure that the card tokens we receive from SmartPay when a user saves a card are encrypted at rest, as part of a defence-in-depth approach. If card tokens are stored in plaintext, an attacker who gains access to the database could potentially steal them and attempt to use them to initiate payments. Unlike Apple Pay payment tokens, which are short-lived and transaction-specific, saved-card tokens are generally long-lived and therefore represent a more persistent security risk. For this reason, I strongly recommend that card tokens are encrypted at the application level before being persisted to the database and decrypted only when they are required for a payment operation. The encryption keys should not be stored in the application configuration or database. They should be securely managed in a Key Vault, with appropriate access controls and key rotation enabled. The encrypted token should also be associated with the version of the key used to encrypt it. When keys are rotated, the application should remain able to decrypt data encrypted with previous key versions. Existing tokens can then be re-encrypted using the new key version, either through a background process or progressively as they are accessed.

I also did a bit of research and ApplePay can only be tested using a physical IOS device. Please refer to this guide <https://developer.apple.com/apple-pay/sandbox-testing/> To save time, we may ask Intellias support for testing ApplePay.

Another point we need to investigate is about the merchant id. This document <https://developer.apple.com/documentation/passkit/offering-apple-pay-in-your-app> mentions it has to be passed in the payment request that the app should craft in order to show the ApplePay sheet.

```
let paymentRequest = PKPaymentRequest()
paymentRequest.paymentSummaryItems =
    paymentSummaryItems
paymentRequest.merchantIdentifier =
    Configuration.Merchant.identifier
paymentRequest.merchantCapabilities =
    .capability3DS
paymentRequest.countryCode = "US"
paymentRequest.currencyCode =
    "USD"
paymentRequest.supportedNetworks =
    PaymentHandler.supportedNetworks
paymentRequest.shippingType =
    .delivery
paymentRequest.shippingMethods =
    shippingMethodCalculator()
paymentRequest.requiredShippingContactFields = [.name, .postalAddress]
#if !os(watchOS)
paymentRequest.supportsCouponCode = true
#endif
```

Remember we have 2 merchant ids and the one to use depends on the user that logged on. We need to check if this could constitute a problem.

Seb

Sebastiano Serri
Integration Architect

EMAIL sebastiano.serri@starkbuild.co.uk
STARK Building Materials UK Limited

